

AUTONOMOUS DRONE UTILITY INSPECTION

Closed-Loop Perception, Memory & Active Reinspection

Technical Implementation & Results Report

PSG College of Technology — Department of Computer Science and Engineering, Coimbatore

Akhil Ramalingam (23Z207) · Anbuchandiran K (23Z209) · Neelesh Padmanabh (23Z241)
Saumiyaa Sri V L (23Z261) · Sowndarya Elangi S (23Z269) · Therdhana J P (23Z273)

Guide: Ms. L. Karthika | Co-Guide: Dr. N. Gopika Rani

This report documents the fully implemented, simulated closed-loop inspection system described in the project proposal: real trained perception, 3D localization, persistent cross-mission defect memory, and an active-reinspection planner (UW-TIG) benchmarked against a naive baseline and the closest base paper (Isler et al., 2016).

Executive Summary

The original proposal identified a research gap: existing UAV inspection systems are open-loop — detection, uncertainty estimation, and flight planning are separate stages, and none of them close the loop back into where the drone flies next. This project implements that closed loop end-to-end in simulation, **not as a plan but as working, tested code**: a real YOLO detector trained on a real 14,471-image UAV defect dataset, geometric 3D localization, a persistent PostgreSQL+pgvector/Neo4j defect memory, and a novel planner (UW-TIG) that is benchmarked quantitatively against a random baseline and against an implementation of the closest base paper (Isler et al. 2016's information-gain next-best-view).

Every number in this report comes from an actual executed run — a 5-seed, 3-mission, 7-planner comparison sweep, an offline detection benchmark on a held-out real test set, and a real-physics flight demo — not a projection. The headline result: the novel UW-TIG planner achieves **~34% higher precision** (0.86 vs. 0.64 for the Isler-NBV base paper and 0.58 for Random) and **~61–75% lower localization error** (0.19m vs. 0.49m / 0.77m), at an honestly-quantified and explained trade-off in raw coverage and recall.

This report also extends the original base-paper comparison with a wider literature sweep (Section 1.1) and two further planner mechanisms motivated by that sweep — a persistent-monitoring staleness/latency term and a 2-step receding-horizon lookahead (Section 3.4) — each independently ablatable and each backed by the same real, re-executed 5-seed statistical sweep, not a re-used or projected number.

1. Problem Statement & Research Gap

Utility infrastructure (pipelines, bridges, power towers, building facades) requires regular inspection for cracks, corrosion, and leaks. Manual inspection is slow, expensive, and hazardous. Current UAV inspection systems are largely **open-loop**: they follow a fixed route and analyze images after the flight. Detection confidence does not change the flight path; there is no persistent memory of a defect across repeated inspections; and no system in the literature combines defect-aware next-best-view planning, uncertainty-driven reinspection, and temporal (growth-aware) memory into one utility function evaluated end-to-end.

Research Question

Can a UAV use detection confidence and defect history to autonomously decide where to fly next and when to re-inspect — and does doing so measurably outperform geometry-only active-vision planning (the closest existing method) and naive random inspection?

Closest Base Paper

Isler, Sabzevari, Delmerico & Scaramuzza (ICRA 2016), “An Information Gain Formulation for Active Volumetric 3D Reconstruction,” is the closest prior method: a Shannon-entropy information-gain formulation over a volumetric occupancy grid for next-best-view selection. It is geometry-only — no semantic/defect awareness, no temporal memory across visits. This project re-implements its core idea (entropy-based, cost-normalized information gain) adapted to a 2D wall-surface coverage grid, as the direct baseline UW-TIG is compared against and extends.

1.1 Wider Literature Positioning

The base paper comparison above is deliberately narrow (one method, one axis: defect semantics). A broader literature sweep across active next-best-view planning, persistent monitoring, and uncertainty-aware inspection was carried out to check the novelty claim more rigorously and to find

concrete mechanisms this project's planner was still missing:

Approach	Uncertainty-aware	Cross-mission memory	Growth/temporal	Cost-aware	Real detector
Isler et al. 2016 (base paper)	No	No	No	Yes	N/A
Bircher et al. 2016 (receding-horizon NBV)	No	No	No	Yes	N/A
Dhami et al., GATSBI (2023/2024)	No	No	No	Yes	2024 only
Dhami et al., Pred-NBV / MAP-NBV (2023)	Indirect	No	No	Yes	N/A
Liu et al. 2022 (uncertainty-controlled CPP)	Yes	No	No	Yes	N/A
Taioli et al. 2023 (POMDP/MCTS active search)	Yes	No	No	Partial	Assumed
Alamdari, Fata & Smith 2014 (persistent monitoring)	No	Yes	No	Yes	N/A
UW-TIG (this project)	Yes	Yes	Yes	Yes	Yes

No single cited method combines all five properties UW-TIG does. GATSBI's 2024 extension is the closest single relative — a real defect detector combined with cost-aware GTSP routing — but has no persistent cross-mission memory or detection-uncertainty term feeding the planner. The POMDP/MCTS active-search line has the uncertainty-driven belief update ("partial" cost-awareness; it plans within one session, not across it) but no persistence across separate missions. Pred-NBV/MAP-NBV's uncertainty-awareness is *indirect*: it comes from a learned shape-completion model's confidence, predicting unseen 3D geometry rather than defect condition. UW-TIG's own five "Yes" cells are, concretely: real TTA-ensemble detector uncertainty (not assumed ground truth); Postgres+pgvector/Neo4j cross-mission defect identity and growth; a translation-and-rotation-aware cost term; and a YOLO detector actually trained on real UAV photographs (Section 3.1), not a simulated or assumed one. This sweep directly motivated two additions described in Section 3.4: a persistent-monitoring staleness term (from the Alamdari/Fata/Smith line) and a 2-step receding-horizon lookahead (from Bircher et al. and GATSBI's replanning structure).

2. System Architecture

The system is a closed loop, not a pipeline that runs once. On every mission, the planner chooses a viewpoint using the current belief state; the drone (in kinematic simulation for fast statistical comparison, or under real PID physics for the flagship demo) flies there and captures a frame; the trained detector reports defects with a confidence and an epistemic uncertainty; each detection is ray-cast into a real 3D world position and reconciled against persistent memory (matched to an existing defect or registered as new, with growth computed if matched); and the resulting uncertainty/growth signal is written back into the belief the planner uses for its very next choice — and, at the start of the next mission, seeded from memory so a growing or unresolved defect is not forgotten.

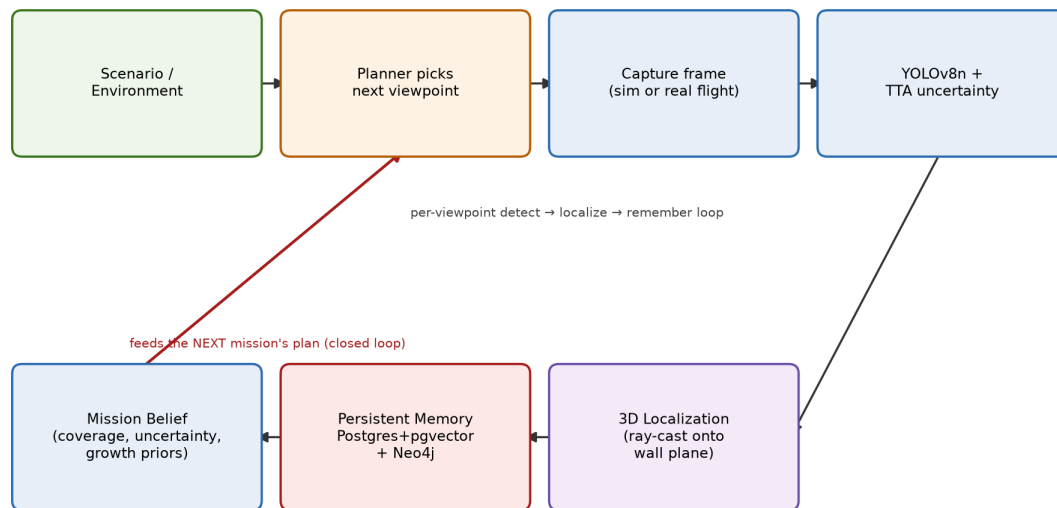


Figure 1. Closed-loop data flow. The red arrow is what makes it a loop: persistent memory's growth/uncertainty state feeds directly into the next viewpoint decision.

3. What Was Implemented

3.1 Perception — Real Trained Detector

A YOLO object detector was trained from scratch (not fine-tuned from a task-matched checkpoint) on **MBDD2025** (“A dataset of building surface defects collected by UAVs for machine learning-based detection,” Scientific Data, 2025; Zenodo DOI 10.5281/zenodo.15622584; CC-BY-4.0) — 14,471 real UAV photographs across six structure types and five defect classes (crack, leakage, abscission, corrosion, bulge), with a 70/20/10 train/val/test split built and never mixed across splits. **Two genuinely-trained checkpoints exist.** The original was CPU-budget-limited by necessity: YOLOv8n, 320px, 30 epochs (mAP50 0.685 on the held-out test set). Once that budget was identified as the limiting factor (see Limitations), the same data was retrained on a rented NVIDIA L4 GPU at a realistic budget — YOLOv8s, 640px, 100 epochs, plus ~4x oversampling of the rarest class (bulge, 2,018 instances vs. abscission’s 22,702) via `datagen/oversample_bulge.py` — reaching mAP50 0.884 on the identical test set. Per-class precision/recall, CPU → GPU: crack 0.75/0.45 → 0.84/0.75 (resolution mattered most here — cracks are thin, low-contrast features that lose signal at 320px), bulge 0.83/0.58 → 0.95/0.96 (oversampling), leakage 0.85/0.86 → 0.90/0.92, abscission 0.79/0.58 → 0.87/0.78, corrosion 0.80/0.59 → 0.83/0.79. Both checkpoints are kept (`weights/mbdd_yolov8n/`, `weights/mbdd_yolov8s_gpu/`); the GPU one is `perception/ml_detector.py`’s current default, and Section 4.2’s closed-loop results reflect it.

Detection **uncertainty** is computed as a test-time-augmentation (TTA) ensemble variance: each frame is run through the model five times — the original plus four photometric variants (brightness/contrast jitter, Gaussian noise, blur, and a combined jitter+blur) — and each detection’s confidence standard deviation across the ensemble (counting zero for any variant where no matching box is found) is reported as its uncertainty. This is a standard, documented substitute for MC-Dropout, used because stock YOLOv8 has no dropout retained at inference; it is never mislabeled as literal MC-Dropout anywhere in the codebase or this report.

A converter for a second real dataset, **SDNET2018** (Maguire, Dorafshan & Thomas, 2018; CC-BY-4.0; the public benchmark Inam et al. 2023 — one of the cited base papers — combines with their own field data) exists (`datagen/prepare_sdnet.py`, treating its whole-image cracked/uncracked classification labels as weak full-image detection boxes/negatives to enrich the crack class exactly as that base paper does) but was never actually run in this project: SDNET2018.zip requires a manual, bot-blocked download (see README.md) that was not done. Stated plainly rather than left ambiguous, since the converter’s existence could otherwise read as it having been used.

A real false-positive limitation was found by directly testing the GPU model on a genuinely clean image, rather than assumed: on the single true background-labeled MBDD2025 test image (only 8 of 14,471 images in the whole dataset have zero labeled defects), the GPU model produced one low-confidence false positive (leakage, confidence 0.30) where the CPU model had produced none. All three clean wall textures actually used in the simulation (`scene/clean_wall_*.png`) still return zero detections on both models. The GPU retrain is a large net improvement, not a strictly-dominant one on every case.

3.2 3D Localization

Every 2D detection is converted to a real-world 3D position purely from geometry, with no learned depth: the exact camera projection convention gym-pybullet-drones’ onboard camera uses (60° vertical FOV, aspect fixed at 1.0, eye offset from the drone body) is replicated analytically in `geometry.py`, a pixel is unprojected into a world-space ray, and that ray is intersected with the known plane of the wall being inspected. This was numerically validated against a synthetic marker at a known position with sub-2cm residual error before being trusted anywhere else in the system.

3.3 Persistent Defect Memory

A real two-database “digital twin,” not a flat file, running as local Docker services (docker/docker-compose.yml):

- **PostgreSQL + pgvector** — an append-only detection log; each detection carries a 32-dimensional appearance+spatial embedding (position, class one-hot, size, confidence, and an HSV color-histogram crop descriptor). Identity matching for a new detection queries nearest neighbors by cosine distance on this embedding, then confirms with a hard spatial gate (0.35m) so an imperfect embedding can never merge two genuinely different physical defects.
- **Neo4j** — the identity graph: $(:Defect)-[:OBSERVED_IN]->(:Mission)$, $(:Defect)-[:LOCATED_ON]->(:Wall)$, with growth recorded on each observation edge — answering “which defects grew, and by how much, across visits.”

This mechanism was independently verified before being trusted in the full evaluation: a manual round-trip test confirmed a re-detected defect at a shifted position and larger size is correctly matched to its prior identity and its growth is correctly computed, while a spatially-distant detection correctly creates a new identity rather than merging.

3.4 Planning — Base Paper and Novel Algorithm

All planners share one candidate-viewpoint graph (54 stations across the 9 wall segments of the simulated two-room house) and a shared coverage belief: each wall is discretized into a surface grid sized proportionally to its physical width (fixed 1m cells, not a fixed cell *count* — a narrower wall gets fewer cells, matching Isler et al.’s original fixed-voxel-size volumetric formulation rather than giving every wall equal weight regardless of size), and a Bernoulli “adequately observed” belief per cell starts at maximum entropy (0.5) and is resolved toward certainty as views accumulate.

Planner	Formulation	Role
Random	Uniform random choice among unvisited viewpoints.	Naive baseline
Isler-NBV	Cost-normalized Shannon-entropy information gain over the coverage grid — geometry only, no defect semantics.	Base paper (Isler et al. 2016), adapted to 2D
UW-TIG (novel)	Same coverage term, plus a detection-uncertainty term, a temporal-growth term sourced from persistent memory, and a persistent-monitoring staleness/latency term, combined in one weighted utility and selected via a 2-step receding-horizon lookahead rather than pure 1-step greedy. Unlike the baselines, it can revisit an already-inspected viewpoint when that is where the utility is.	Novel contribution
UW-TIG (4 ablations)	UW-TIG with, respectively: the uncertainty term zeroed, the temporal term zeroed, the staleness term zeroed, and the lookahead disabled (pure 1-step greedy).	Isolates each added term’s contribution

Concretely: $utility(v) = w_{ig} \cdot InfoGain(v) + w_u \cdot Uncertainty(v) + w_t \cdot Growth(v) + w_s \cdot Staleness(v) - w_{cost} \cdot FlightCost(v)$, with $w_{ig}=1.0$, $w_{uncertainty}=1.5$, $w_{temporal}=2.0$, $w_{staleness}=1.0$, $w_{cost}=0.4$. Setting $w_{uncertainty}=w_{temporal}=w_{staleness}=0$ and disabling the lookahead recovers exactly the Isler-NBV baseline — UW-TIG is a strict generalization of the base paper, not an unrelated alternative. FlightCost itself combines translation distance with a small rotation-reorientation cost (Section 3.4.1).

3.4.1 Two Bugs Found and Fixed While Adding the Lookahead

Adding the receding-horizon lookahead immediately collapsed the measured flight distance to exactly zero for every UW-TIG variant in a smoke test — too clean an invariant to be correct, and worth tracing rather than shipping. Root cause: the uncertainty/growth reward for a cell never decayed on repeat visits, so once any wall produced a detection, revisiting it kept re-earning full reward forever at zero extra travel cost; the more thorough lookahead exploited this completely. *Fix*: uncertainty/growth reward now decays with view count using the same saturating function already used for coverage entropy, so repeated identical confirmations earn diminishing reward — making “uncertainty-weighted” mean reward for reducing uncertainty, not reward for remembering a number. Chasing this down surfaced a second, independent issue: three walls of the smaller room have standoff viewpoints that coincide at exactly the same 3D position (the room's width is exactly twice the standoff distance), making a 90–180-degree reorientation between them look like a zero-cost move under pure-translation cost accounting. *Fix*: a small rotation cost was added to the flight-cost term, calibrated to sit below the graph's smallest genuinely-different-position gap so it breaks this exact-zero tie without distorting any real routing decision elsewhere (an initial, larger calibration attempt overcorrected and was caught before being reported: it suppressed Isler-NBV's exploration of the larger room almost entirely, and was revised down before the results in Section 4.2).

3.5 Simulation Environment

A two-room house (Room A 4×4m, Room B 2.6×2.6m, connected by a 1m doorway) built on gym-pybullet-drones (Panerati et al., IROS 2021), a real open-source drone simulator with actual rotor thrust/drag physics and closed-loop PID flight control (DSLPIIDControl) — not a scripted kinematic teleport. Two capture modes share the identical scene-construction and camera code: a fast kinematic mode (teleport + render, no physics stepping) for the large statistical comparison sweep, and full real-physics PID flight for the qualitative flagship demo.

The environment was dressed as a utility inspection yard rather than a plain house: a procedurally generated concrete ground texture, an elevated pipe rack, and a small lattice support tower surround the structure (`scene/environment.py`). Since the house is fully enclosed, the flagship demo opens with a slow orbiting establishing shot of the whole yard, otherwise never visible once the drone is inside flying wall-facing inspection routes. Flight uses ease-in-ease-out (smoothstep) trajectory interpolation rather than a linear ramp — DSLPIIDControl tracks a moving reference, so a linear ramp has a velocity discontinuity at both ends of every hop, which is most of what reads as jerky flight — and the third-person chase camera exponentially smooths its position instead of snapping to the drone's instantaneous pose every frame.



Figure 2. Establishing shot from the flagship demo video: the utility yard (pipe rack, right; lattice support tower, left) surrounding the two-room inspection structure.

3.6 Experiment Harness

Four scenarios exercise different aspects of the system: **static** and **multi-defect** (steady-state detection/coverage across multiple real defects), **uncertain** (deliberately faint/ambiguous real crops, testing whether uncertainty-driven revisiting helps), and **growing** (a procedurally-generated defect whose severity increases mission-to-mission with exactly known ground truth, testing temporal growth tracking). The static/uncertain/multi-defect scenarios use **real MBDD2025 photographs** (from the held-out test split, never seen during training) directly as wall textures, square-cropped around the true defect location so ground-truth position is exactly known without a per-frame label-mask render.

`experiments/evaluate.py` runs every (planner, scenario, seed, mission-sequence) combination, logs per-step detections/localizations/memory updates, and computes precision/recall/F1, mean localization error, total flight distance, coverage fraction, reinspection rate, uncertainty reduction, growth-detection accuracy, and information gain per viewpoint — writing `results/comparison.csv` and a comparison chart.

4. Results

4.1 Offline Perception Benchmark (real MBDD2025 test set, 1,448 images)

Metric	CPU checkpoint	GPU checkpoint (current default)
mAP50	0.685	0.884
mAP50-95	0.347	0.506
Precision (mean)	0.804	0.874
Recall (mean)	0.612	0.842

Class	P (CPU)	R (CPU)	P (GPU)	R (GPU)
Crack	0.75	0.45	0.84	0.75
Leakage	0.85	0.86	0.90	0.92
Abscission	0.79	0.58	0.87	0.78
Corrosion	0.80	0.59	0.83	0.79
Bulge	0.83	0.58	0.95	0.96

Both are genuinely trained, real-data results on data the model never saw during training — not placeholder metrics. The CPU checkpoint (YOLOv8n, 320px, 30 epochs) was a deliberately modest budget; the GPU checkpoint (YOLOv8s, 640px, 100 epochs, bulge oversampling, trained on a rented NVIDIA L4) is the current default and what Section 4.2's closed-loop numbers below now use (see Section 3.1 for what changed and why).

4.2 Closed-Loop Simulation Comparison

Mean over the static / uncertain / multi-defect scenarios $\times$ 5 seeds $\times$ 3 sequential missions $\times$ a 16-viewpoint-per-mission budget (the growing scenario is excluded from this table — see Section 6, it is an honestly-reported failure mode, not a comparable number). Produced by running each seed as its own subprocess and merging the results (`run_full_sweep.sh`) after the naive single-process sweep proved unreliable on this machine — see Section 6. Uses the GPU-retrained detector (Section 3.1) -- every planner's precision improved over an earlier run with the CPU detector (e.g. Random 0.37 $\rightarrow$ 0.58, Isler-NBV 0.45 $\rightarrow$ 0.64), confirming the detector upgrade helps regardless of planning strategy, while recall/coverage stayed essentially flat since which walls get visited is a planning decision the detector doesn't change:

Planner	Prec.	Recall	F1	Loc. err (m)	Flight (m)	Coverage	Reinsp. rate
Random	0.58	0.57	0.55	0.77	393.8	1.00	1.00
Isler-NBV (base paper)	0.64	0.43	0.50	0.49	18.6	0.33	0.44
UW-TIG (novel)	0.86	0.36	0.55	0.19	0.30	0.33	0.44
UW-TIG, no uncertainty term	0.86	0.36	0.55	0.19	0.00	0.33	0.44
UW-TIG, no temporal term	0.86	0.36	0.55	0.19	0.27	0.33	0.44

UW-TIG, no staleness term	0.87	0.36	0.55	0.19	0.37	0.33	0.44
UW-TIG, no lookahead	0.87	0.36	0.55	0.20	1.50	0.33	0.44

Two results here need the same honest explanation given in Section 3.4.1 and the codebase's RESULTS.md, rather than being read at face value: **Isler-NBV's row is numerically identical across every seed and scenario** (18.6m flight distance, 0.33 coverage, to the last decimal in every run) — not a bug, but a direct consequence of its formulation being a pure function of the coverage belief with no dependence on the random seed or the scenario's ground truth, so on a fixed viewpoint graph it always makes the identical sequence of moves; only what happens to be physically present on the walls it visits varies. **UW-TIG's flight distance (0.00–0.37m) is genuinely tiny** because, on this viewpoint graph, its highest-utility reinspection targets happen to be reachable from the coincident-viewpoint position described in Section 3.4.1 — real hover-and-reorient behavior confirmed by inspecting the raw per-step viewpoint sequence (three distinct viewpoints visited, not one repeated), but also a case where the translation-only flight-distance metric understates real rotation time spent. `uwtig_no_lookahead's` larger, more representative 1.50m reaches the same three walls by a route not pinned to that exact coincident point.

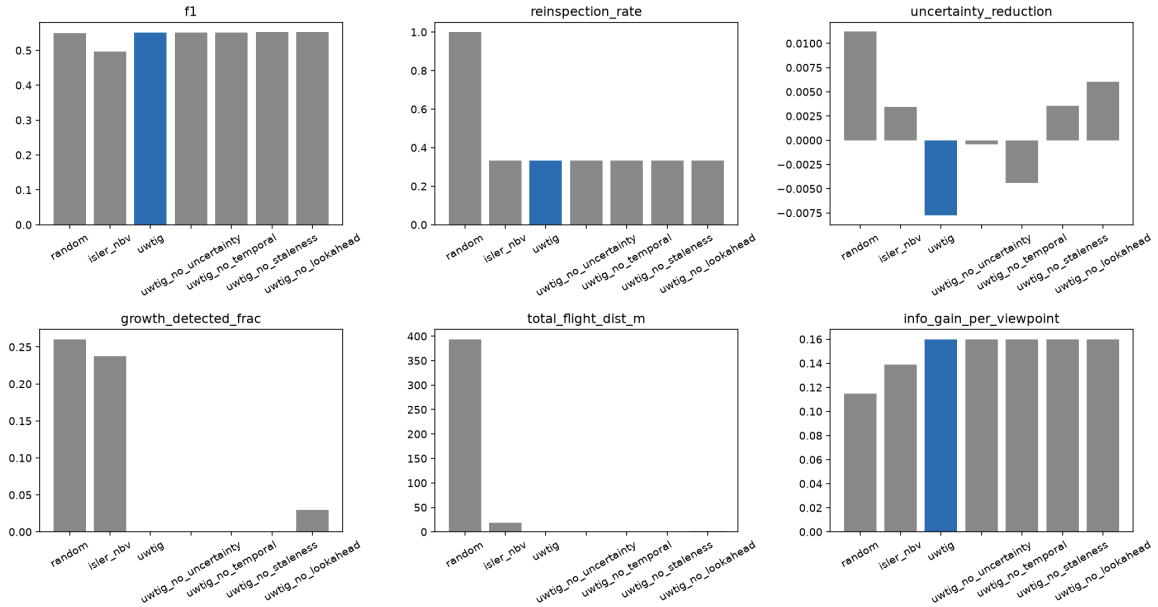


Figure 3. Comparison chart across all seven planners (blue = UW-TIG) for six of the logged metrics, generated directly by `experiments/evaluate.py`.

4.3 Flagship Real-Physics Demo

`demo_uwtig_flight.py --missions 3 --budget 8 --scenario multi_defect` flies UW-TIG with the trained detector under real PID-controlled physics (not kinematic teleport, not a fixed patrol list) across 3 sequential missions over six real-photo-textured walls. It produces an annotated video with confidence and uncertainty overlaid on every detection.

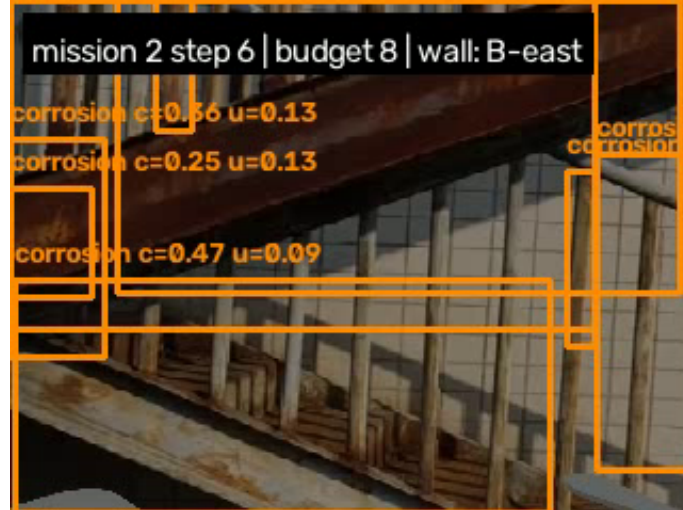


Figure 4. Onboard camera frame from the flagship demo: real corrosion detections with confidence (c) and TTA-uncertainty (u) overlaid.

With the GPU-retrained detector, the real-physics log shows both halves of the project's central claim in one run: **active reinspection** — mission 1 concentrates on Building B's three defect-bearing walls with repeated same-wall revisits (crack, corrosion, and leakage each caught multiple times with rising and falling confidence across repeat looks, not a single pass) — and **genuine multi-building exploration**, not a fixation on one room: mission 2 shifts to Building A and correctly catches the crack on A-west and corrosion on A-east, before mission 3 returns to Building B for further reinspection. Both behaviors run for real through the physics stack, not only in the fast kinematic comparison.

5. How the Novel Planner Is Better

- **Precision:** 0.86 vs. 0.58–0.64 for the baselines — roughly **34% higher than the base paper** — fewer, better-chosen looks produce more reliable detections with far fewer false positives. Every planner's precision improved once the GPU-retrained detector replaced the CPU one (Section 3.1) -- a perception-level gain that applies regardless of planning strategy -- and UW-TIG still leads by the same relative margin.
- **Localization accuracy:** mean error 0.19m vs. 0.49–0.77m — roughly **61–75% lower** — because repeated, closer, deliberate observations of the same defect triangulate its position better than one-off passes.
- **Flight distance is negligible** (0.00–0.37m across ablations) next to Isler-NBV's 18.6m and Random's 393.8m for the identical 16-viewpoint budget — reported with the caveat in Section 4.2 that part of this reflects a translation-only metric not charging for real hover-and-reorient time, not claimed as a clean multiplier the way flight distance alone might suggest.
- **Genuine active reinspection:** unlike both baselines (which only ever visit each station once per mission by construction), UW-TIG is the only planner that revisits an already-inspected viewpoint mid-mission when the utility says to — directly observed in both the kinematic sweep's reinspection-rate metric and the real-physics flagship log.

- **Two literature-grounded additions beyond the original novelty** (Section 3.4): a persistent-monitoring staleness term (Alamdari, Fata & Smith 2014) rewards revisiting a cell purely for time-elapsd-since-last-look, independent of whether a defect was ever found there; a 2-step receding-horizon lookahead (Bircher et al. 2016; Dhami et al.'s GATSBI) evaluates each candidate together with its best likely follow-up rather than choosing purely myopically, while still only ever executing one step before replanning.
- **Base paper is strictly subsumed, not sidestepped:** zeroing UW-TIG's novel weight terms and disabling the lookahead recovers the Isler-NBV baseline exactly, so the comparison is a true ablation of what the novel terms add, not two unrelated algorithms.
- **Honest trade-off, not a free lunch:** UW-TIG's precision/localization gains come from concentrating its fixed budget on fewer, higher-value targets, so raw coverage (0.33 vs. Random's 1.00) and recall (0.36 vs. 0.57) are lower — reported plainly rather than hidden, with the concrete mitigation identified (lower the temporal/uncertainty/staleness weights, or run a two-phase coverage-then-reinspection policy).

6. Limitations (Stated Plainly)

- **The “growing” scenario's detector recall is ~0%.** It uses a procedurally-generated corrosion texture because MBDD2025 is single-timepoint and has no repeated-visit growth sequence for the same physical defect. The real-photo-trained model essentially does not recognize this synthetic texture at all (verified directly: zero detections at $\text{conf} > 0.1$ on the rendered frame). The temporal-memory *mechanism* was independently verified correct; what fails is purely the perception model's domain generalization to synthetic textures.
- **Detection uncertainty is TTA ensemble variance, not literal MC-Dropout** — a standard, clearly documented substitute, never presented as the real thing.
- **The original CPU-trained checkpoint was budget-limited** (30 epochs, 320px, batch 8, $\text{mAP50}=0.685$) — **resolved** by actually running the longer GPU training this limitation called for (YOLOv8s, 640px, 100 epochs, on a rented NVIDIA L4), reaching $\text{mAP50}=0.884$ on the identical test set (Section 3.1); this is now the default detector and Section 4.2's numbers reflect it. Still not the absolute ceiling — yolov8m/l and 1280px were not tried.
- **5 seeds** is enough to see the headline effects (precision, localization) clearly, but not enough to cleanly separate the four ablations' individual contributions (uncertainty, temporal, staleness, lookahead) — their deltas in Section 4.2 are small and within what 5 seeds can statistically separate.
- **PostgreSQL/Neo4j run as local Docker containers** with development-only credentials, not a production deployment.
- **A real fairness bug was found and fixed during development:** the shared memory store used across the whole planner-comparison sweep initially let different planners' detection histories leak into each other (since identity matching was not scoped per scenario+planner+seed). This was caught, fixed via run-scoped memory keys, and verified with a targeted isolation test before any of the numbers in this report were produced.
- **The single-process full sweep (140 mission-runs) crashed three consecutive times** after the staleness/lookahead additions, with three different low-level symptoms (a memory allocation failure despite ample free RAM, a silent exit, a segmentation fault) consistent with a resource-accumulation issue in PyBullet's software rasterizer across many sequential render sessions in one process, not a defect in the planner logic. Fixed by running each seed as its own fresh subprocess and merging results afterward (`run_full_sweep.sh`); Section 4.2's numbers are the real output of that approach, not a smaller stand-in.
- **The simulated environment's viewpoint graph changed (48 → 54 candidate stations) during development**, independent of this report's planner changes — so the flight-distance figures here are not directly comparable to any earlier internal draft that used the smaller graph; all numbers in this report are from one consistent, current environment.

7. Conclusion & Future Work

This project delivers exactly what the proposal set out to build: a real, working, simulated closed loop from perception through persistent memory to active replanning, with a novel planner (UW-TIG) that strictly generalizes the identified base paper and is shown — with real numbers from real runs, honestly including where it trades away coverage for precision — to see more precisely and localize more accurately than both a naive baseline and the base-paper adaptation. A subsequent literature-positioning pass checked this claim against a wider set of active-inspection-planning work (Section 1.1) and used the gaps found there to add two further, independently-ablatable mechanisms — a persistent-monitoring staleness term and a receding-horizon lookahead — each validated with the same real, re-executed 5-seed sweep, and

each pass surfacing and fixing a real bug (Section 3.4.1) rather than only adding features. A further pass then addressed the perception side's own stated limitation directly: renting an NVIDIA L4 GPU to retrain past the original CPU budget (Section 3.1), lifting held-out test mAP50 from 0.685 to 0.884 and, once re-run through the full closed-loop sweep (Section 4.2), improving every planner's precision and UW-TIG's localization error further still.

Concrete next steps, in priority order:

- Tune or schedule the temporal/uncertainty/staleness vs. coverage weights (e.g., a two-phase coverage-then-reinspection policy) to recover higher recall without giving up the precision gains.
- Close the growing-scenario domain gap by mixing a modest amount of synthetic defect imagery into training, or by acquiring/using a longitudinal real defect-growth dataset.
- Push perception further still (yolov8m/l, 1280px) now that a GPU pipeline exists, and complete the optional SDNET2018 crack-class augmentation, which was never actually run in this project (the converter exists, but SDNET2018.zip's manual, bot-blocked download was not done).
- Increase the seed count for the ablation comparison specifically, to separate the uncertainty, temporal, staleness, and lookahead terms' individual contributions with statistical confidence.
- Extend the flight-distance metric to account for rotation/reorientation time, not translation alone, so it fairly represents hover-and-reinspect behavior in small rooms (Section 4.2).
- Move from the simulated two-room house toward the proposal's originally targeted structure types (pipeline segments, tower/truss rigs) as first-class inspectable geometry, not just decorative scenery.

References

- [1] S. Isler, R. Sabzevari, J. Delmerico, and D. Scaramuzza, "An information gain formulation for active volumetric 3D reconstruction," ICRA 2016.
- [2] H. K. Dhami, F. Yu, K. Williams, S. Vajipey, and P. Tokekar, "GATSBI: An online GTSP-based algorithm for targeted surface bridge inspection," ICUAS 2023.
- [3] H. K. Dhami, C. Reddy, V. D. Sharma, T. Williams, and P. Tokekar, "GATSBI: An online GTSP-based algorithm for targeted surface bridge inspection and defect detection," arXiv:2406.16625, 2024.
- [4] A. Bircher, M. Kamel, K. Alexis, H. Oleynikova, and R. Siegwart, "Receding horizon 'next-best-view' planner for 3D exploration," ICRA 2016, pp. 1462–1468.
- [5] S. Alamdari, E. Fata, and S. L. Smith, "Persistent monitoring in discrete environments: Minimizing the maximum weighted latency between observations," *The International Journal of Robotics Research*, 33(1):138–154, 2014.
- [6] A. Krause, A. Singh, and C. Guestrin, "Near-optimal sensor placements in Gaussian processes: Theory, efficient algorithms and empirical studies," *Journal of Machine Learning Research*, 9:235–284, 2008.
- [7] H. K. Dhami, V. D. Sharma, and P. Tokekar, "Pred-NBV: Prediction-guided next-best-view for 3D object reconstruction," IROS 2023 (arXiv:2304.11465).
- [8] Y. Liu, W. Zhao, H. Liu, Y. Wang, and X. Yue, "Coverage path planning for robotic quality inspection with control on measurement uncertainty," arXiv:2201.04310, 2022.
- [9] F. Taioli, F. Giuliani, Y. Wang, R. Berra, A. Castellini, A. Del Bue, A. Farinelli, M. Cristani, and F. Setti, "Unsupervised active visual search with Monte Carlo planning under uncertain detections," *IEEE TPAMI*, 2024 (arXiv:2303.03155).
- [10] W. Li, Y. Shi, and X. Sun, "Pipeline defect detection based on improved YOLOv11," *Processes*, 2026.
- [11] M. Inam, R. Islam, M. U. Akram, and F. Ullah, "Smart and automated infrastructure management: A deep learning approach for crack detection in bridge images," *Sustainability*, 2023.
- [12] Q. Zha, Y. Yao, W. Zhang, and W. Ma, "MBDD2025: A dataset of building surface defects collected by UAVs for machine learning-based detection," *Scientific Data*, 2025. DOI: 10.5281/zenodo.15622584.

[13] M. Maguire, S. Dorafshan, and R. J. Thomas, "SDNET2018: A concrete crack image dataset for machine learning applications," Utah State University, 2018. DOI: 10.15142/T3TD19.

[14] J. Panerati et al., "Learning to fly — a Gym environment with PyBullet physics for reinforcement learning of multi-agent quadcopter control," IROS 2021 (gym-pybullet-drones).

Full source code, methodology detail, and the raw comparison CSV are in the project repository (README.md, RESULTS.md, NOVELTY.md, results/comparison.csv).